REED command line flags and syntax summary

March 16, 2026

1 REED command line flags summary

The following summary was generated automatically by using `reed -summarise`, running REED version 2.3. Compiled 12:10:24, Mar 16 2026.

`-#`
: show comments in the REED file, useful if comments are used as reminders.

`-basename <path/file.ext>`
: set pathname for generated files (replacing `.ext` with `.gv`, `.html`, `.pdf`, etc).

`-c`
: show weakly connected components.

`-allcolors`
: list all colors that are available for highlighting.

`-colors`
: list all colors used.

`-colors+`
: lists all colors like `-colors` but also gives the meanings of all colors used.

`-F`
: highlight flags in graph drawing — unfortunately due to a GraphViz bug, this breaks up any groups.

`-f`
: show textual descriptions of flag colors in REED drawing.

`-flags`
: show all flag definitions.

`-g`
: generate a GraphViz `.gv` file — GraphViz is the default option if nothing else chosen — See `https://graphviz.org` for details.

`-go`
: as `-g` but also open the `.gv` file.

`-h`
: generate an interactive `.html` REED file.

`-ho`
: as `-h` but also open the `.html` file.

`-ids`
: show full names and IDs in the REED graph.

`-insert <text>`
: insert this text to process before the next file.

`-json`
: generate a JSON `.js` file with all information from the GraphViz diagram.

`-keywords`
: list all keywords used.

`-l`
: generate a Latex REED file — also generates some useful Latex definition files.

`-m`
: generate a Mathematica `.nb` file representing the REED graph as a series of expressions.

`-n`
: show node IDs in graph drawing.

`-o`
: open generated files automatically.

`-pdf`
: generate a `.pdf` file representing the REED graph.

`-pick <color>`
: restrict generated files to just this color.

`-pick <keyword>`
: restrict generated files to notes with this keyword. Keyword can be abbreviated: use `...` so `xyz...` matches keywords or phrases starting `xyz`.

`-pick+ <color>`
: same as `-pick` but also explains this color meaning.

`-raw`
: start in raw mode (skipping text until a start tag) — only use `-raw` with `-tags` flag.

`-rules`
: summarise HTML-Latex rules.

`-s`
: list nodes and their full names, sorted by node ID.

`-ss`
: list nodes and their full names, sorted by full names.

`-sss`
: list nodes and their full names, sorted by both node ID and full names.

`-sep`
: draw a separator line before processing any files (useful with `-watch`).

`-sig`
: show REED file MD5 signatures.

`-summarise`
: put Latex overview of REED syntax and command line flags to standard output.

`-svg`
: generate a `.svg` file — representing the REED graph.

`-syntax`
: summarise REED syntax.

`-t`
: transpose node numbering — swap row and column node numbers.

`-tags <start> <end>`
: only process REED information written between these tags — you can change tags between files — and also set tags within a REED file by: `tags <start> <end>`.

`-transit`
: list transit nodes and nodes with no influence.

`-v`
: verbose mode.

`-version`
: state the version number of the REED command.

`-w`
: what versions are used in these files? — helpful to know if using the `v=` flag.

`-watch`
: run REED when any used file changes (nice with `-o` flag).

`-x`

generate an `.xml` file — representing all REED data for import into other applications.

`--`

treat all further parameters as filenames — if you want to have no restrictions on filenames as they otherwise cannot be flags.

2 REED syntax summary

The following summary was generated automatically by using `reed -summarise`, running REED version 2.3. Compiled 12:10:24, Mar 16 2026.

```
# Hash # indicates comment to end of line
# Hash # is also used for comments in this syntax summary

### syntax notation...
### blanks can be put between all syntactic items and used freely for layout

x ::= y # define non-terminal x to be y
x | y   # x or y
{ x }   # grouping in syntax definitions
x+      # at least one x. Short for: x | xx | xxx | ...
x++     # at least one x separated by spaces. Short for: x | x x | x x x |...
[ x ]   # optional x

### REED syntax...

id ::= { letter | digit | underline | dot }+ | Cstring | Herestring

Cstring ::= "C string, \n for newline, \\ for backslash etc"

Herestring ::= <<< arbitrary-terminator
                   characters (no escapes needed) over any number of lines
                   ends with same arbitrary-terminator text
                   at start of line
arbitrary-terminator

properties ::= { id1 : id2 }++
  Used in note id to define property id1 of id as id2

### narrative strings used in notes are translated to HTML and/or Latex
### use -rules to summarise the automatic HTML/Latex translations
```

narrative strings can include:

```
<html> ...          # following text is copied to HTML, but skipped in Latex
<latex> ...         # following text is copied to Latex, but skipped in HTML
<both> ...          # following text is mixed basic HTML and Latex
<input> filename    # insert the text contents of file filename into the string
[[[id]]]            # replace with id name and make a cross-reference link
                    # ([[[id]]] makes links in both HTML and Latex)
```

```
# reserved words can be used as ids if written out as strings
# otherwise, an id xyz and a string "xyz" are equivalent
# so check (a command) is reserved, but "check" can be used arbitrarily
# string notations also allow more characters, like newlines, to be used
```

the following are definitions of non-terminals:

```
arrow ::= influences: | -> | <-
color ::= aqua | black | blue | fuchsia | gray | green
        | lime | maroon | navy | olive | orange | purple
        | red | silver | teal | white | yellow
# note that color names cannot be used as ids
# NB reed -allcolors lists all supported colors

id-or-star ::= id | *
node-list ::= id | ( id++ )
node-or-arrow ::= id | id { arrow id }+
node-or-arrow-list ::= node-or-arrow | ( node-or-arrow++ )
document-property ::= title | author | abstract | date
                    | introduction | conclusion
```

the following are REED statements
all terminals below are reserved words
reserved words can use lower and upper case letters freely

`check id1 => id2`
  Check there is a path from `id1` to `id2`
`cycle node-list`
  Confirms that these nodes are intended to be on a cycle, and do not report them
`direction { LR | RL | TB | BT }`
  Primary direction of graph is left-right, right-left, top-bottom, or bottom-top
`definitions.html id`
  Insert `id` at start of `body` in HTML file; multiple `definitions` concatenate
`definitions.latex id`

Insert `id` at start of Latex file
multiple `definitions` concatenate

`definitions.latexend id`
Insert `id` at end of Latex file (e.g., for references)
multiple `definitions` concatenate

`document-property id`
Specify document properties. Details append when given multiple times
(See also `definitions` for more properties)

`group node-or-arrow-list is id`
The nodes are grouped, and the group is named `id`

`highlight color [ cascade ] is id`
Define meaning of color as `id`
If cascade is specified, apply to all nodes the `color` influences

`highlight id [ cascade ] is color`
Highlight node `id` with `color`
Optional `cascade` colors all nodes `id` influences

`layout [ TOC ] ( { id ( id++ ) }++ )`
Draw a graph with a structured layout, with optional TOC (Theory of Change) styling

`node-or-arrow [ is id ]`
Create nodes or arrows, with optional display name `id`

`norefs`
Nodes without reference numbers will not be reported as errors

`note node-or-arrow-list [ properties ] [ is id1 [;] ] id3`
Provide narrative note `id3` for the nodes or arrows, optionally naming them `id1`

`numbering ( { ( id-or-star++ ) }++ )`
Make node references row.column numbers following the grid layout

`ref id1 is id2`
Set the reference of node `id1` to be `id2`

`style node-or-arrow-list is id`
Set style of nodes or arrows to a Dot specification,
which can contain further REED style names (see `style.new`)

`style.cycle id`
Define style to highlight cycles

`style.default id`
Define default style for nodes and edges

`style.new id1 is id2`
Define `id1` as a style, using Dot syntax for `id2`
`id2` can contain further REED style names (but not recursively)

`tags id1 id2`
Only process REED code between `id1` (begin) and `id2` (end)

`version id`
Subsequent REED code is version `id` (see flag `v=value`)

`;`
semicolons are empty statements
Semicolons can be placed freely between statements,
and in `note` statements to aid legibility.

The order of commands

The order of commands does not matter, except:
`author`, `definitions.html`, `definitions.latex`, `definitions.latexend`
which all concatenate in the order given,
and `tag` and `version`, which (if more than one) are executed in the order given.

Technical note on style names and spaces:

Style names defined by `style.new id` can be used anywhere in styles,
which means `id` may need to be surrounded by spaces to be recognised
e.g., `xidy` has to be written `x id y` for `id` to be recognised,
or of courses can be written `x;id;y` etc.
Any spaces around using `id` will be stripped off,
so if you do need spaces around a named style
those spaces have to be put in the definition of `id`.